

Heterogeneous Preference Training

Dynamic Opponents Improve Preference Training in Heterogeneous LLM Cross-Play

Vasyagina Ekaterina, MIPT · Qwen2.5-0.5B-Instruct vs. SmolLM2-1.7B-Instruct on GSM8K

TL;DR

What? We study heterogeneous online preference training: Qwen and SmolLM answer the same GSM8K prompt, a verifier converts their answers into a preference pair, and both policies are updated with DPO.

Why? In self-play, comparisons are drawn from one model family or a fixed reference. A **live heterogeneous opponent** changes the preference-pair distribution and exposes different reasoning and formatting errors.

How? The comparison is controlled by matching exposure for both policies:

$$\Delta_{\text{replay}} = \Delta_{\text{pairs}} = \Delta_{\text{updates}} = 0.$$

Result. At checkpoint 1000, CrossPlay reaches validation win rates 0.340/0.516 for Qwen/SmolLM and higher verifier rewards than the corresponding self-play baselines.

Background: From RLHF to Games

Preference optimization usually starts from pairs (y^+, y^-) and updates a policy relative to a reference. DPO makes this practical by replacing PPO-style reward optimization with a contrastive objective.

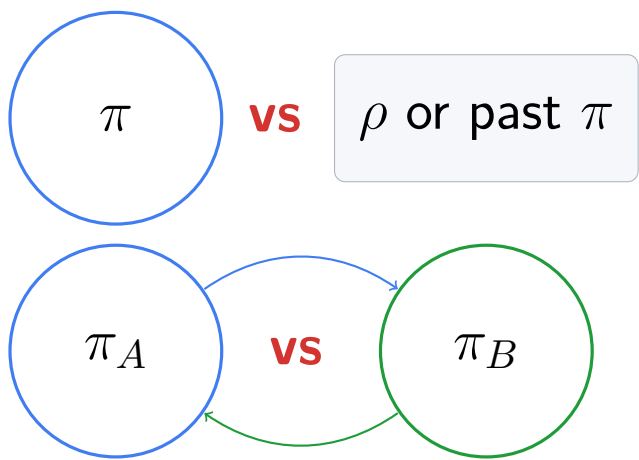
NLHF reframes alignment as a game over pairwise preferences:

$$p_{\phi}(x, y, y') = P(y \succ y' \mid x),$$

and seeks a policy that is robust against competing policies. Online IPO and SPPO show that online preference sampling and self-play can be connected to Nash-style equilibria.

Question. Does changing the live opponent model improve the preference data used for training, when data volume and update counts are held fixed?

Why Dynamic Opponents?



Static self-play: the opponent distribution changes slowly or not at all.

CrossPlay: two different live models generate changing, asymmetric comparisons.

Hypothesis. A heterogeneous opponent changes the online preference distribution $q_t(y^+, y^- \mid x)$, producing comparisons that are less tied to one model's own error profile.

Control. Replay entries, selected pairs, and optimizer steps are equalized; the intended experimental variable is the opponent distribution.

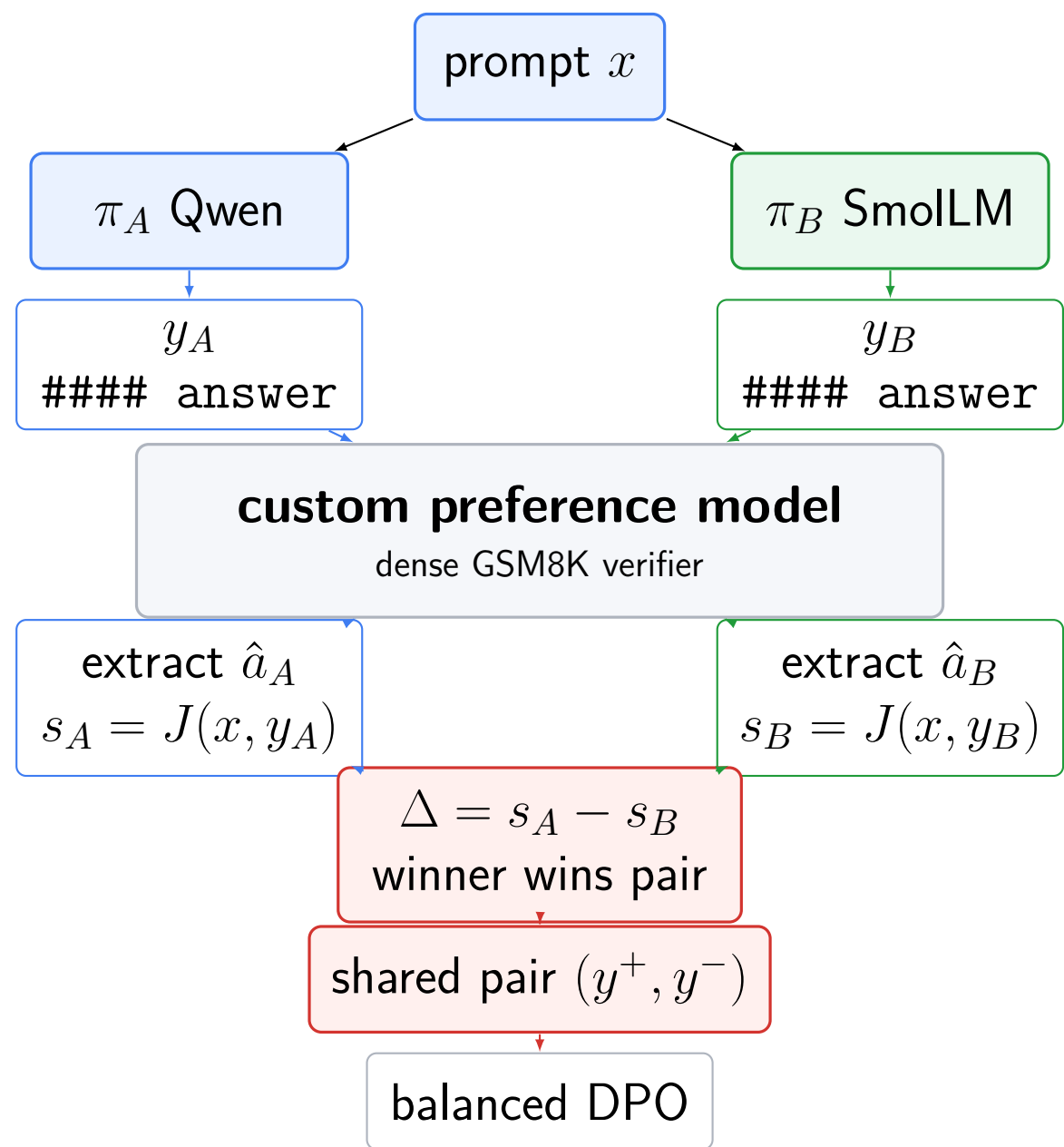
Verifier-based Preference Construction

For each prompt x , both models generate answers y_A, y_B . A dense GSM8K verifier extracts the final answer $\hat{a}_i$ and assigns a score s_i .

$$\begin{aligned} r_i^{\text{bad}} &= \min(0.30, \max(0.02m_i, 0.05h_i) + 0.03h_i + 0.20h_iq_i), \\ s_i &= \mathbf{1}\{\hat{a}_i = a^*\} + \mathbf{1}\{\hat{a}_i \neq a^*\}r_i^{\text{bad}}, \\ q_i &= \left(1 + \frac{|\hat{a}_i - a^*|}{\max(|a^*|, 1)}\right)^{-1}, \quad q_i = 0 \text{ if nonnumeric}, \\ m_i &= \mathbf{1}\{\text{#### parsed}\}, \quad h_i = \mathbf{1}\{\hat{a}_i \text{ extracted}\}, \\ y^+ &= \arg \max_{y_i \in \{y_A, y_B\}} s_i, \quad y^- = \{y_A, y_B\} \setminus y^+. \end{aligned}$$

The higher-scoring answer becomes the preferred response, producing the DPO pair (y^+, y^-) .

Heterogeneous CrossPlay



At training step t , a prompt $x_t \sim \mathcal{D}$ is answered by both live policies, $y_i \sim \pi_i^t(\cdot \mid x_t)$ for $i \in \{A, B\}$. The verifier constructs $(y^+, y^-) = C(x_t, y_A, y_B)$, and the same preference pair is used to update each policy against its own frozen anchor ρ_i :

$$z_i = \beta \left[\log \frac{\pi_i(y^+ \mid x_t)}{\pi_i(y^- \mid x_t)} - \log \frac{\rho_i(y^+ \mid x_t)}{\rho_i(y^- \mid x_t)} \right], \quad \mathcal{L}_i = -\log \sigma(z_i).$$

Replay entries, DPO pairs, and optimizer steps are matched across A and B .

Experimental Setup

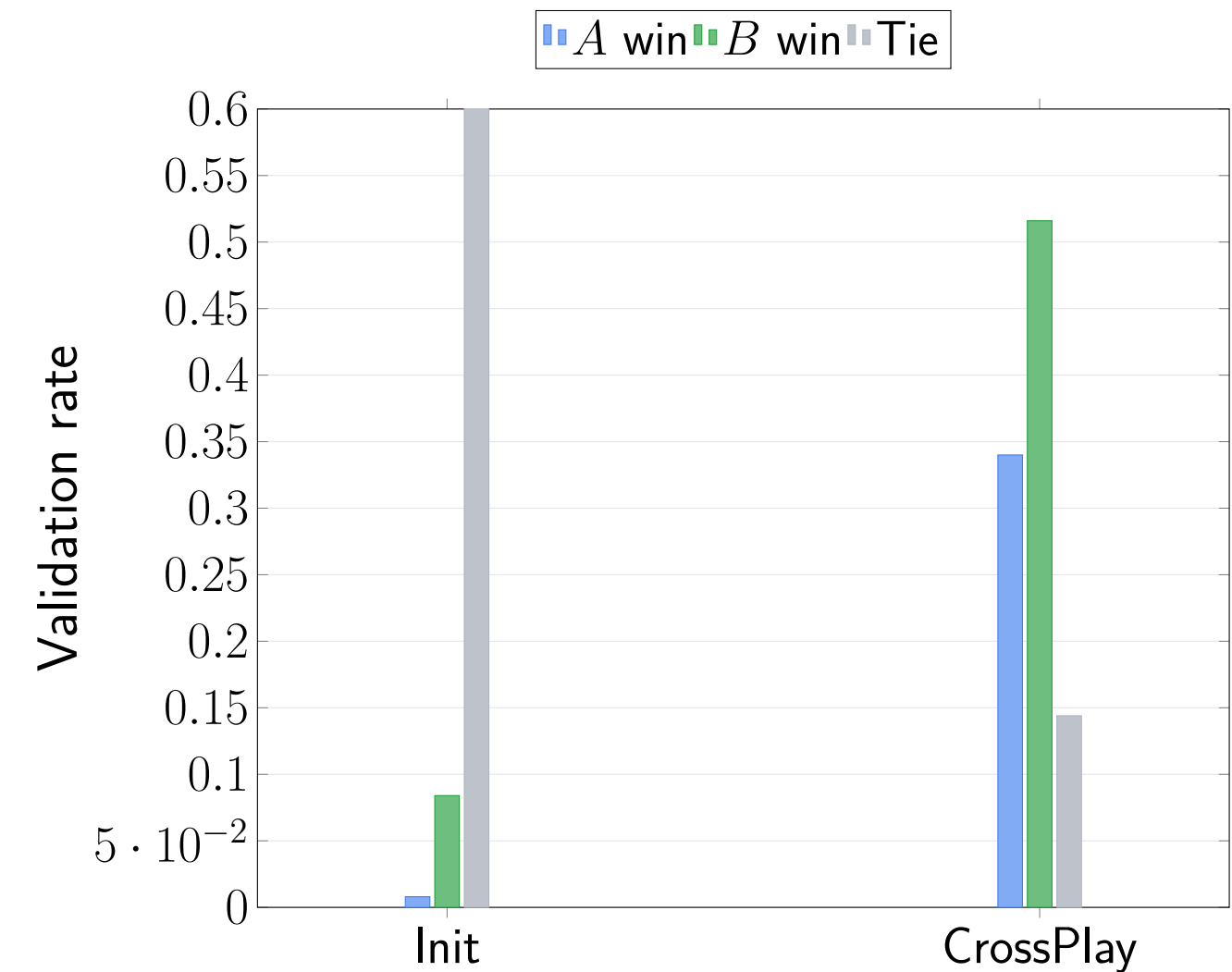
Models	Qwen2.5-0.5B-Instruct, SmolLM2-1.7B-Instruct
Task	GSM8K math reasoning
Training subset	2000 examples
Update	sigmoid DPO, $\beta = 0.1$
Batching	train batch = 1, micro batch = 1
Responses	128 tokens, final answer as #### <answer>
Reward	1.0 exact; otherwise dense score with format, answer, and numeric-proximity terms, capped at 0.30
Checkpoints	every 200 update steps

Controlled Findings

- Accounting control.** Replay entries, selected DPO pairs, and update steps are equalized for both policies.
- Training signal.** CrossPlay improves validation win rate and dense verifier reward over initialization; it also exceeds the provisional self-play baselines.
- Scope.** The current system uses a dense GSM8K verifier rather than a learned human-preference model, so it is an NLHF-motivated approximation rather than full NLHF.

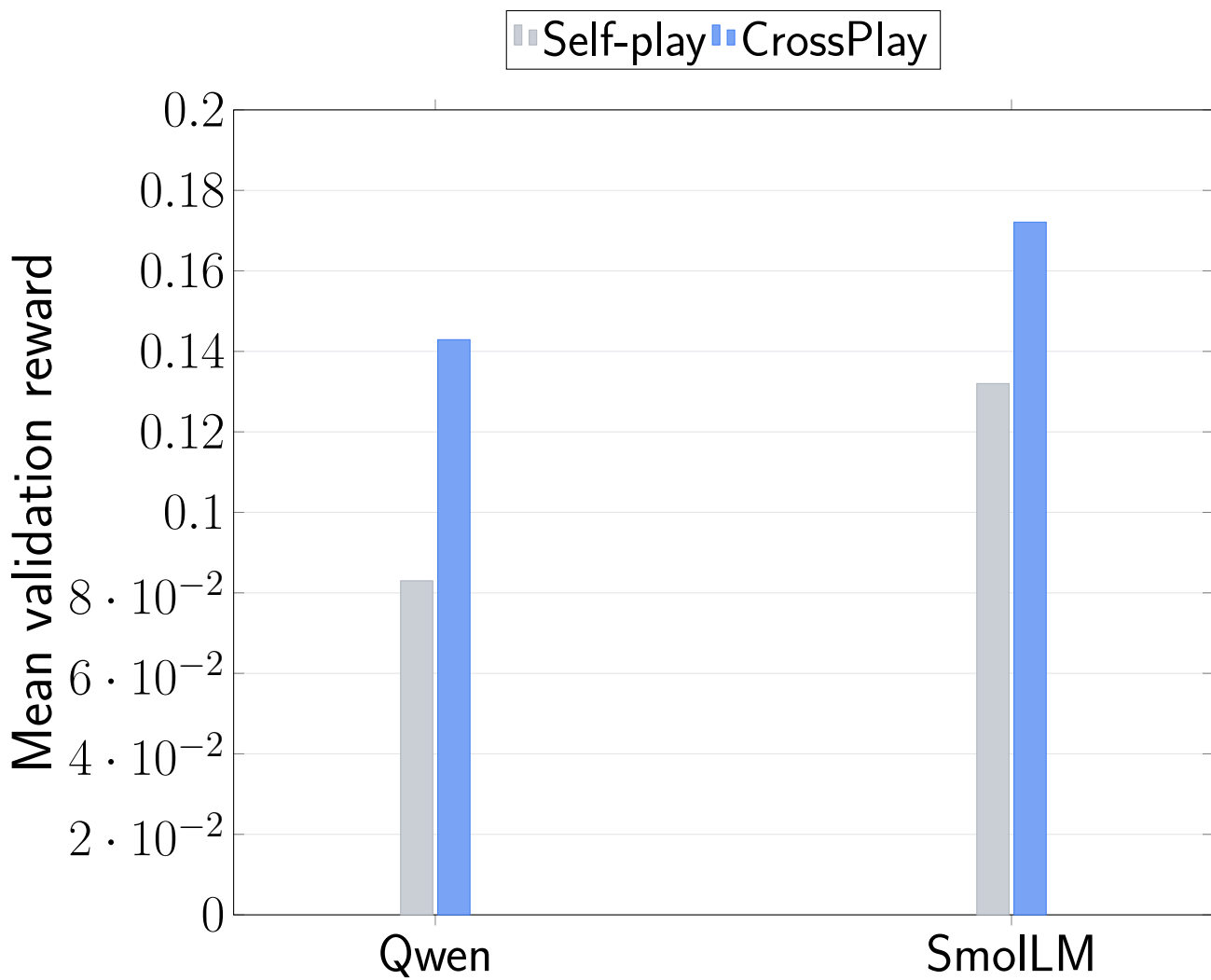
Experimental Results

Initialization vs. CrossPlay



Stage	A win	B win	Tie
Init, real	0.008	0.084	0.908
CP@1000, real	0.340	0.516	0.144

CrossPlay vs. Self-Play



Model	Metric	Self [†]	CP	Δ
Qwen	win	0.285	0.340	+0.055
Qwen	reward	0.083	0.1429	+0.0599
SmolLM	win	0.465	0.516	+0.051
SmolLM	reward	0.132	0.1721	+0.0401

Balance Audit

Quantity	A – B	Removed
Replay	0	data volume
DPO pairs	0	pair count
Updates	0	optimizer exposure

Interpretation. The main accounting confound is removed: neither policy receives more replay data, DPO pairs, or optimizer steps. Under this audit, the measured contrast is same-family self-play versus heterogeneous online CrossPlay.

Evaluation protocol. Validation reports win/tie rates and dense verifier rewards on the same held-out GSM8K prompts; self-play numbers are marked as provisional estimates.